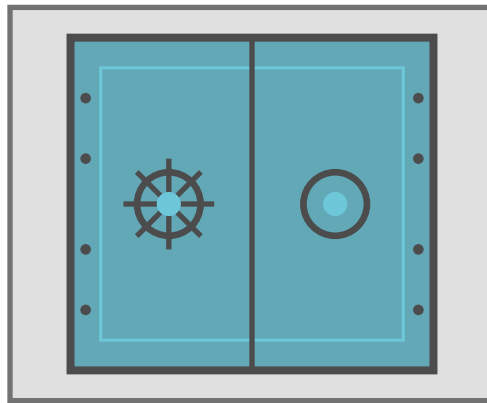


Criptografía y Seguridad

Seguridad en aplicaciones

Seguridad en la empresa



Apunte integral — Teoría + Práctica

Clase 10 · Lectura recomendada: *NIST SP 800-207* (caps. 2 y 3)

Instituto Tecnológico de Buenos Aires (ITBA)

*Documento elaborado a partir de las transcripciones de la clase teórica (Prof. Leandro Suárez) y de las diapositivas oficiales de la teórica y la práctica. Esta clase **no está en Bishop**: está enfocada en el panorama profesional contemporáneo de la seguridad en organizaciones.*

1 Introducción y contexto

Esta clase cierra el bloque de **Seguridad en aplicaciones** pasando de la teoría (autenticación, control de acceso, principios de diseño, vulnerabilidades) a la **visión profesional**: cómo una empresa real adopta distintas estrategias para *protegerse* de un abanico amplio de amenazas, equilibrando seguridad y negocio.

Dónde se sitúa esta clase

“Esto de la clase que vamos a ver hoy no está en Bishop. [...] está más enfocado en lo que es el panorama contemporáneo, lo profesional: cómo se maneja esto en un ambiente profesional de una organización, una corporación, incluso una Pyme con buenas prácticas.”

— dicho en clase

La lectura recomendada para profundizar es el estándar **NIST SP 800-207** (Zero Trust), capítulos 2 y 3.

Tríada CIA — el marco de toda la seguridad (recordatorio)

Toda política de seguridad busca *preservar* tres propiedades sobre la información:

- **Confidencialidad**: guardar la info en secreto; acceso sólo a autorizados.
- **Integridad**: confiabilidad de los datos. Se distingue *integridad de datos* (no se alteran) e *integridad de origen* (proviene de quien dicen).
- **Disponibilidad**: poder usar la información cuando se la necesita.

El propósito de todo lo que veremos hoy es, en el fondo, **forzar el cumplimiento de la tríada CIA dentro de la organización**, tanto de forma **proactiva** (barreras, controles, evitar exponer vulnerabilidades) como **reactiva** (qué medidas tomar una vez que ya nos atacaron).

1.1 Una mirada del temario desde la práctica: la certificación CISSP

La clase práctica abre presentando la certificación profesional de referencia en este campo: **CISSP** (*Certified Information System Security Professional*, de ISC2). Sus **8 dominios de conocimiento** son un excelente mapa mental de *toda* la materia de seguridad de la empresa:

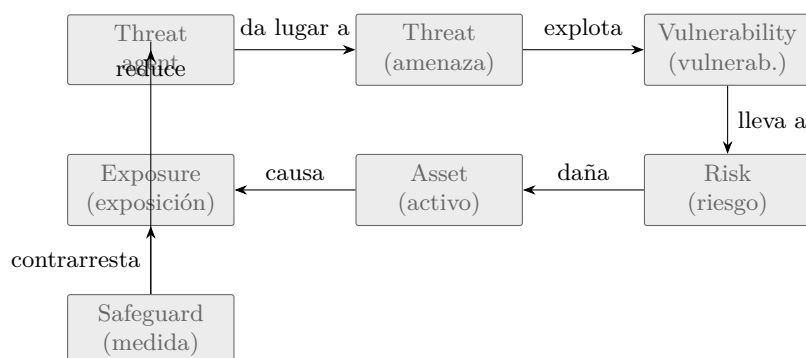
Los 8 dominios de CISSP (aporte de la práctica)

1. Gestión de Seguridad y riesgos
2. Seguridad de activos
3. Arquitectura e ingeniería de la seguridad
4. Seguridad de comunicaciones y redes
5. Gestión de identidades y accesos (IAM)
6. Evaluación y pruebas de seguridad
7. Operaciones de seguridad
8. Seguridad del desarrollo de software

Casi todos los temas de esta clase (gobernanza, gestión de riesgos, IAM/PAM, evaluación de seguridad, desarrollo seguro) caen dentro de alguno de estos dominios.

1.2 Vocabulario de seguridad (definiciones de la práctica)

La práctica fija un vocabulario preciso con el clásico diagrama de Bishop (*threat agent* → *threat* → *vulnerability* → *risk* → *asset*). Conviene tenerlo claro porque toda la gestión de riesgos se apoya en él.



Definiciones clave

Amenaza (threat)

Peligro potencial asociado a la explotación de una vulnerabilidad.

Vulnerabilidad

Falta o debilidad en una medida implementada.

Riesgo (risk)

Probabilidad de que un agente explote la vulnerabilidad y produzca efectos en el negocio.

Exposición

Exposición a pérdidas.

Medida / control (safeguard)

Mecanismo para reducir el riesgo.

Tipos de control (aporte de la práctica)

Los controles de seguridad se clasifican en tres familias, y la **defensa en profundidad** consiste justamente en combinarlas en capas concéntricas:

- **Administrativos** (políticas, procedimientos, capacitación).
- **Técnicos (lógicos)** (firewalls, cifrado, autenticación).
- **Físicos** (puertas, cámaras, control de acceso a instalaciones).

Políticas vs. Mecanismos (aporte de la práctica)

Distinción fundamental que recorre toda la materia:

- **Política:** define *qué acciones son seguras (permitidas)* y *qué acciones son inseguras (prohibidas)*.
- **Mecanismo:** procedimiento, herramienta o método para *garantizar el cumplimiento* de la política. Los mecanismos pueden ser de tipo: **prevenir, disuadir, detectar, corregir, recuperar, compensar**.

2 Gobernanza (Governance)

Gobernanza de seguridad

Conjunto de **procesos, políticas, procedimientos y controles** implementados para **gestionar y mitigar los riesgos** de la organización, estableciendo **responsabilidades y roles** dentro de la entidad para garantizar la protección de la información.

La analogía del profesor: la gobernanza dentro de una empresa es como el **gobierno** de un Estado.

La analogía del gobierno

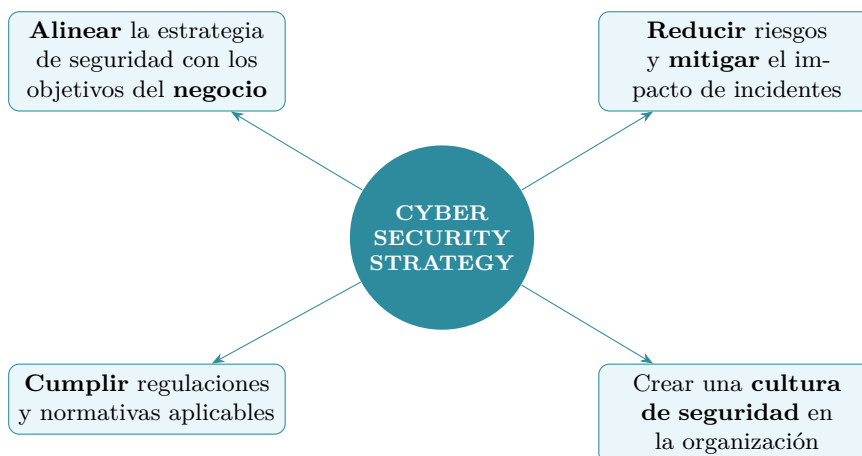
“El gobierno expide leyes y después es responsable de alguna forma de hacer que esas leyes se cumplan. Dentro de una empresa existe algo bastante análogo: la gobernanza está dada generalmente dentro de un equipo de seguridad, que emite las reglas, decide qué reglas hay que tomar y se hace responsable de que se cumplan.”

— dicho en clase

2.1 Importancia en la organización

- Asegura la confidencialidad, integridad y disponibilidad de la información.
- Protege contra amenazas cibernéticas y violaciones de seguridad.
- Fomenta la **confianza de clientes y stakeholders**.

2.2 Objetivos principales de la gobernanza



Cuidado: no atentar contra el negocio (aceptación psicológica)

La estrategia de seguridad **no debe ir en contra de cómo la empresa gana dinero**. Si se ponen demasiadas trabas, los empleados *buscan otros caminos* y terminan atentando contra la seguridad. Es el problema de la **aceptación psicológica** visto en clases anteriores.

“Si yo le pongo muchas barreras a un empleado [...] se van a bajar todos los datos en un CSV, lo van a publicar en Drive o en algún lugar, y van a empezar a elaborar ahí. Y esto lo digo porque sé que ha pasado.”

— dicho en clase

Por qué importa la cultura: el eslabón más débil

“¿Cuál es el eslabón más débil en un sistema? Los usuarios, la gente que interactúa con esos sistemas terminan siendo los más expuestos.”

— dicho en clase

Por eso la gobernanza debe generar una **cultura de seguridad**: campañas de concientización, campañas de phishing simuladas y capacitación, sobre todo a quienes trabajan con tarjetas de crédito, datos personales o información crítica.

Regulaciones que condicionan la estrategia (ejemplos del profe): empresas que cotizan en bolsa deben *informar todo incidente de seguridad* (caso contrario hay consecuencias penales); el manejo de **datos personales** exige protegerlos o se arriesgan *multas millonarias* ante un robo.

3 Gestión de Riesgos (Risk Management)

Gestión de riesgos

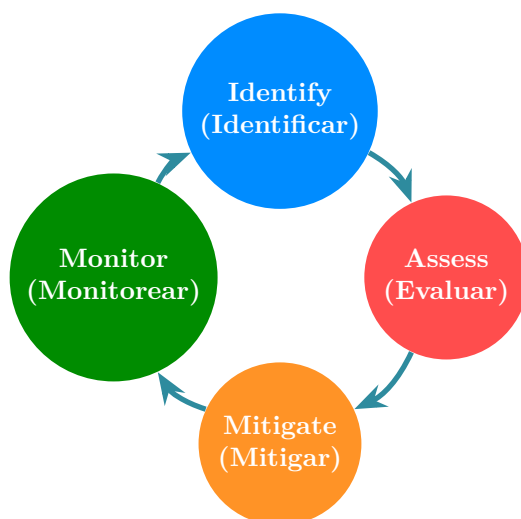
Identificación, evaluación y control de amenazas potenciales para el negocio, con el objetivo de **minimizar el impacto** de los incidentes de seguridad en la operación.

Por qué la seguridad se gestiona como un riesgo

“Antes —y todavía se sigue haciendo— la seguridad se gestionaba como un riesgo más. Las empresas ya saben de riesgos, saben cómo manejarlos, mitigarlos, eliminarlos. Entonces la seguridad se trataba como algo más, y la verdad que funciona, sobre todo en las grandes corporaciones.”

— dicho en clase

La gestión de riesgos es un **ciclo continuo** de 4 etapas — nunca termina, porque los riesgos evolucionan, decrecen, se transforman o aparecen nuevos.



3.1 Identificación del riesgo

Primero se identifican los **activos críticos para el negocio** (¿qué puede afectar al ciclo de negocio?) y luego los **riesgos potenciales** sobre ellos.

Activos críticos	Riesgos potenciales
• Datos (empleados, clientes, comerciales, financieros, proveedores) • Sistemas (principal activo de las empresas tech) • Infraestructura (servidores, data centers, oficinas físicas) • Personas (las empresas no se mueven solas)	• Malware • Ataques externos • Errores humanos • Sabotaje (amenaza interna) • Ambientales (huracán, incendio)

Riesgos externos a la empresa: caída de una región cloud

Además de los de la slide, el profe remarca los **problemas externos**: proveedores de infraestructura cloud, Internet, DNS...

“Se caía una región de AWS y casi todas las empresas argentinas o de Latam están subidas a esa región [...] se caía PedidosYa, no se podía usar.”

— dicho en clase

Si la empresa sólo tiene contratada esa región, deja de funcionar.

3.2 Evaluación del riesgo (Assess)

- Se identifica la **probabilidad de ocurrencia** y el **impacto** en el negocio.
- Se realizan **evaluaciones externas de seguridad** (Red Team / PenTest).
- Se clasifican los riesgos según su impacto: **Alto / Medio / Bajo**.

La matriz probabilidad × impacto

En la práctica se arman **matrices de probabilidad vs. impacto**: se ataca primero lo de *máxima probabilidad y mayor impacto*, y se barre en diagonal hacia abajo. Hay casos opuestos:

“Es muy poco probable que pase esto, pero si llega a pasar estamos al horno.”

— dicho en clase

Ahí la empresa, como ente que afronta una pérdida económica, decide qué hacer con cada caso.

3.3 Mitigación del riesgo (Mitigate)

En la etapa de mitigación se elaboran **políticas de gestión**, se definen y evalúan **procedimientos de operación y acción ante incidentes**, se diseñan y prueban **planes de acción** y se capacita al personal a todo nivel. Existen **5 estrategias** para tratar un riesgo:



Las 5 estrategias de tratamiento del riesgo

Evitarlo (avoid)

Quitar la causa del riesgo. Suele ser *costoso y complejo* (ej.: arreglar vulnerabilidades en un sistema legacy o migrarlo, con posibles *side effects* que rompen el sistema). Se paga el costo si realmente hay que solucionarlo.

Reducirlo (reduce/mitigate)

Bajar el impacto o la probabilidad. **Es lo más común**. Ejemplo: autenticación débil con sólo password → se le suma un **factor extra (MFA)**; no es tan caro como migrar o dar de baja un sistema.

Transferirlo (transfer)

Delegarlo a un **proveedor**; si hay un problema, responde el proveedor. Ejemplo: poner Cloudflare como *proxy reverso* y punto de entrada de una web expuesta a Internet, aplicando políticas anti-ataque.

Compensarlo (compensate)

No es lo mismo que reducir. Cuando no se puede evitar ni reducir, se **contrarresta el impacto** con un control adicional. Ejemplo: dependencias que podrían tener vulnerabilidades y que no se pueden quitar → **auditar y controlar** esa dependencia para detectar CVEs apenas aparecen.

Aceptarlo (accept)

Asumir el riesgo (típico en riesgos de *baja probabilidad*). A veces tomar otra alternativa es más costoso que aceptarlo. **Siempre se documenta**: el equipo de Risk Management deja constancia de que la empresa decidió aceptarlo.

Riesgo aceptado en la práctica: el DRP que no se ejecuta

A las empresas que cotizan en bolsa se les exige tener un **DRP** (*Disaster Recovery Plan*). Para los bancos digitales y fintech, el **Banco Central** exige que el DRP esté *documentado y probado*. Pero a muchas otras empresas sólo se les exige *tenerlo*:

“Tengo el plan, está aceptado el riesgo, pero no lo ejecuto porque ejecutarlo me sale un montón de plata. Si pasa algo, pasa, y confiamos en que funcione.”

— dicho en clase

Ejecutar un DRP implica tener toda la infra duplicada con todos los enlaces y aplicaciones levantadas, lo cual es muy costoso.

3.4 Monitoreo y Evaluación (Monitor)

- **Elaboración y supervisión de controles y medidas de seguridad.**
- Auditorías internas y externas.
- Simulaciones de ataques e incidentes.

Es la etapa que **cierra el ciclo**: como la organización es un *sistema vivo*, hay que estar ejecutando y revisando esto constantemente. Existen **frameworks** que dan un punto de partida; el profe menciona **NIST** (la organización de estándares de EE.UU., análoga al IRAM en Argentina), que desarrolló frameworks para medir, mitigar y gestionar riesgos de forma segura.

CIS Critical Security Controls v8 (<https://www.cisecurity.org/controls>) — la slide muestra los 18 controles agrupados; los *foundational* (1–8) son: (1) Inventory & Control of Assets, (2) Inventory & Control of Software, (3) Data Protection, (4) Secure Configuration, (5) Account Management, (6) Access Control, (7) Continuous Vulnerability Management, (8) SIEM. Los restantes incluyen (14) Security Awareness & Training, (16) Application Software Security, (17) Incident Response, (18) Penetration Testing.

Conclusión (slide): la gestión de riesgos es **fundamental** para proteger la información; un **enfoque proactivo** reduce la probabilidad y el impacto de incidentes; y la **revisión y adaptación continua** es esencial para enfrentar nuevos desafíos.

4 Políticas de Seguridad

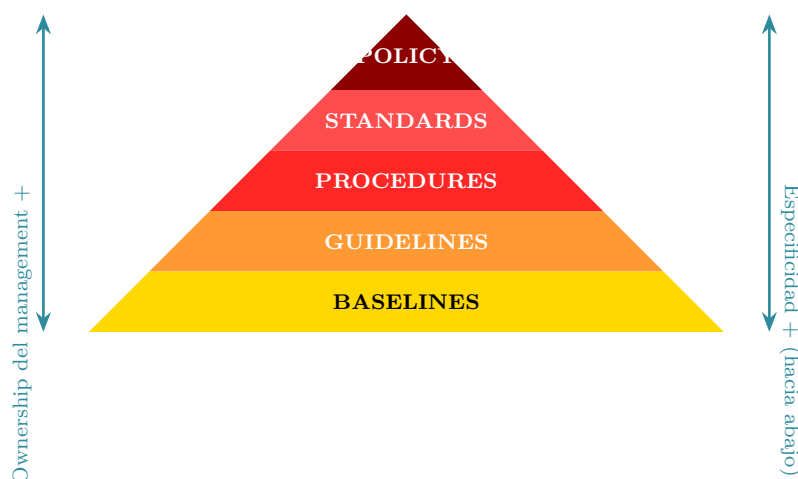
La política como “Biblia” de seguridad

“La política de seguridad es como una especie de Biblia para la seguridad de la empresa. Es un documento bastante extenso y denso que se confecciona una sola vez cada cierto tiempo, se va revisando y lo aprueba el director de seguridad (CISO).”

— dicho en clase

Es muy genérica y abarcativa; por eso luego se **desglosa** en documentos cada vez más específicos hasta llegar al empleado más raso.

La jerarquía de documentos va de *mayor ownership del management / menor especificidad* (arriba) a *menor ownership / mayor especificidad* (abajo). Las dos capas superiores son **mandatorias**; las inferiores, **discrecionales**.



Niveles de la jerarquía de políticas

Policy (Política)

General management statement. Documento genérico y abarcativo de toda la seguridad de la organización. **Mandatorio.**

Standards (Estándares)

Specific mandatory controls. Reglas obligatorias de seguridad que el hardware y el software deben cumplir. **Mandatorio.** Ej.: un estándar de desarrollo seguro.

Procedures (Procedimientos)

Step-by-step instructions. Tareas detalladas paso a paso para lograr algún objetivo.

Guidelines (Guías / directrices)

Recommendations / best practices. Acciones recomendadas para usuarios y personal cuando no puede aplicarse un estándar específico. **Discrecional.**

Baselines

Uniform ways for safeguard implementation. Suelen tomar forma de cursos, capacitaciones y concientizaciones. **Discrecional.**

Estándares de NIST (aporte de la práctica)

La práctica detalla los estándares concretos del **NIST** (*National Institute of Standards and Technology*):

- **NIST 800-53:** *obligatorio* para sistemas de información federales (EE.UU.).
- **NIST CSF 2.0** (*Cybersecurity Framework*): *voluntario* para las organizaciones, basado en riesgos, más flexible y adaptable. Sus funciones son: **Gobernar, Identificar, Proteger, Detectar, Responder, Recuperar**. Distingue *Gobernanza* (cómo se gestiona la organización) de *Gobernanza de seguridad* (cómo se gestiona la seguridad vía políticas y procedimientos).
- **NIST SP 800-207:** define la **Zero Trust Architecture (ZTA)**, con las ideas *Never Trust, Always Verify, Assume breach*. (Lectura recomendada de la clase.)

5 Estrategias de seguridad

Con el tiempo surgieron formas genéricas de *implementar* las políticas de seguridad. Como las políticas de empresas similares se parecen mucho entre sí (

“la política de Mercado Libre debe ser muy parecida a la de OLX o PedidosYa”

— dicho en clase

), aparecieron estrategias reutilizables. Hoy se destacan **dos**: *Defense-in-depth* y *Zero Trust Architecture*.

5.1 Defense-in-depth (Defensa en profundidad)

Defense-in-depth

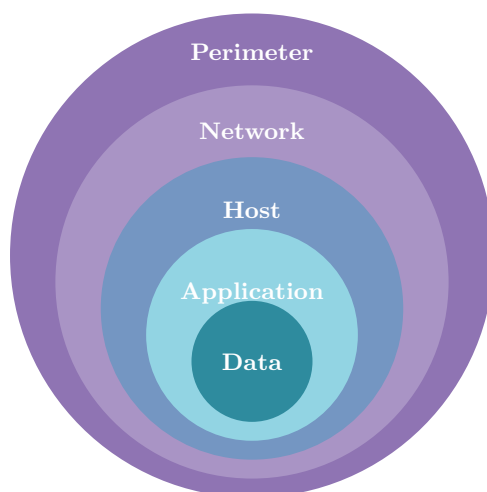
Estrategia que aplica **múltiples capas de defensa** para proteger los activos de la organización. Parte de asumir que **ningún control de seguridad es perfecto** (la seguridad nunca está asegurada al 100%) y busca crear **redundancia**: cada capa está pensada para *contener a la capa que sigue*.

La cebolla

“Esto se piensa como si fuera una cebolla: agarrando capas y metiéndote cada vez más adentro hasta llegar a lo más importante. Si falla la capa de arriba, la contiene la de abajo, y así.”

— dicho en clase

Las **5 capas** (de afuera hacia el activo crítico):



Las 5 capas de Defense-in-depth

Perímetro

Ej.: un **firewall** que deniega llamadas de Internet y sólo acepta de cierta red o VPN.

Red (Network)

Segmentación de red: separar la red corporativa (operaciones) de la red de servidores. Un empleado del call center no debe poder llegar al servidor de ventas.

Host

“**Jardinizar**” (hardening) los hosts: el host tiene un software fijo, inmutable, no modificable —como un contenedor Docker—, optimizado con buenas prácticas.

Aplicación

Buen diseño, buenas prácticas, sin vulnerabilidades (todo lo visto en las clases anteriores de la materia).

Datos

La capa más interna, lo que más importa. Para llegar, un atacante debe atravesar *todas* las capas anteriores: firewall → moverse lateralmente entre redes → vulnerar el host jardinizado → vulnerar la aplicación.

Soluciones por capa (corte de la slide “Outside Threat Protection”). La diapositiva muestra, en abanico, qué herramientas pueblan cada capa, atravesadas por las funciones *Prevention*, *Policy Management*, *Operations*, *Monitoring & Response*:

Capa	Herramientas (de la slide y comentadas por el profe)
Perímetro	Honeypot (deriva al atacante a una “red boba” para entretenerlo), Message Security (anti-virus/anti-malware), Secure DMZs, Perimeter Firewall, Perimeter IDS/IPS.
Red	NAC (Network Access Control: decide si un dispositivo se conecta según IP, MAC, firmware o credenciales), Web Proxy / Content Filtering (ej. Fortinet: bloquea Netflix o sitios maliciosos), Enterprise Message Security, DLP .
Host (Endpoint)	Content Security (anti-virus/anti-malware), Host IDS/IPS, Endpoint Security Enforcement, FDCC Compliance, Patch Management.
Aplicación	WAF (Web Application Firewall: como un firewall pero para tráfico HTTP/HTTPS de una app web; filtra o reemplaza tráfico según reglas), Dynamic App Testing, Database Monitoring/Scanning.
Datos	IAM (Identity & Access Management: autenticación, autorización, gestión de identidad, roles, permisos), PKI, Data Classification, Data Wiping, Data/Drive Encryption, Database Secure Gateway.
Monitoring	SOC/NOC 24x7, CIRT (Incident Reporting/Detection/Response), SIEM , Digital Forensics.

IDS / IPS y DLP

IDS / IPS

Intrusion Detection/Prevention System. Detectan patrones tipo antivirus (por **firmas** o **anomalías**) y deciden si bloquear, alertar o actuar.

DLP

Data Loss Prevention.

“No es que se pierda data dentro de la compañía, sino que no se filtre data hacia afuera.”

— dicho en clase

Escanean para que no se exfiltren datos de tarjeta o personales; si pasa, levantan alertas.

5.1.1 Principios de Defense-in-depth

- **Diversificación:** usar diversas tecnologías y métodos $\Rightarrow$ una sola vulnerabilidad no atraviesa todo.
- **Redundancia:** múltiples capas que se superponen para asegurar protección continua.
- **Segmentación:** dividir la red en segmentos controlados para limitar el acceso no autorizado.

“Hay compañías muy avanzadas donde tu aplicación no ve nada: si sólo tiene que hablar con otra aplicación, sólo tiene visibilidad de esa otra.”

— dicho en clase

- **Monitoreo:** visibilidad de todas las capas y **correlación de eventos** para detectar ataques.

Problemas de Defense-in-depth

- **Altos costos:** de tecnología (muchos proveedores distintos, ≥ 5 presupuestos), de capacitación y operación (no hay “bala de plata”, cada producto es distinto), y de monitoreo (*se recolectan muchos datos pero se les saca poco valor*).

- **No resuelve eficientemente los ataques internos.** Si alguien ya está adentro (robó credenciales de un empleado), es muy difícil darse cuenta: no todas las herramientas son bidireccionales.
- **Difícil de implementar con sistemas distribuidos:** data centers híbridos (propios + terceros), múltiples proveedores de nube, empresas *cloud native*. Esta estrategia fue pensada para la época de *data center / on-premise*.

El intranet “de confianza” que quedó obsoleto

Tradicionalmente, dentro del **intranet** de la empresa se relajaban los controles (menos autenticación, sesiones más largas) para no friccionar al empleado.

“Eso durante mucho tiempo se consideró buena práctica y seguro. Hoy sabemos que estamos mirando otro tipo de estrategias.”

— dicho en clase

Eso da pie al Zero Trust.

5.2 Zero Trust Architecture (ZTA)

Zero Trust Architecture (NIST SP 800-207)

Modelo de seguridad que asume que las amenazas, **tanto internas como externas**, están presentes en la red **en todo momento**. **Objetivo:** proteger la integridad, confidencialidad y disponibilidad de los datos y recursos. **Premisa:** cualquier red interna es tan peligrosa como Internet (“Internet es *Bad Lands*, la tierra de nadie”).

Por qué ninguna red es segura

“Si vos fuiste a Starbucks, te fuiste al baño, dejaste la máquina y vino un chabón con un pendrive y te instaló un exploit/malware, y venís y te conectás... yo no te pido nada, asumo que tu computadora está bien y ya estás inyectando algo dentro de la red.”

— dicho en clase

Con el trabajo remoto es aún peor: no se sabe quién accede. Por eso se considera que **toda red interna es tan peligrosa como Internet** y se aplican adentro los mismos controles que afuera.

Los 3 principios básicos de Zero Trust

1. **Nunca confíes** (*Never Trust*).
2. **Siempre verifica** (*Always Verify*).
3. **Asume que has sido atacado** (*Assume breach*).

Cómo se relaja en la práctica: SSO / federación

En teoría *cada* componente debería forzar la verificación, pero pedir 20 logins a 20 aplicaciones haría que el empleado se queje. En la práctica se relaja con **federación de identidad**:

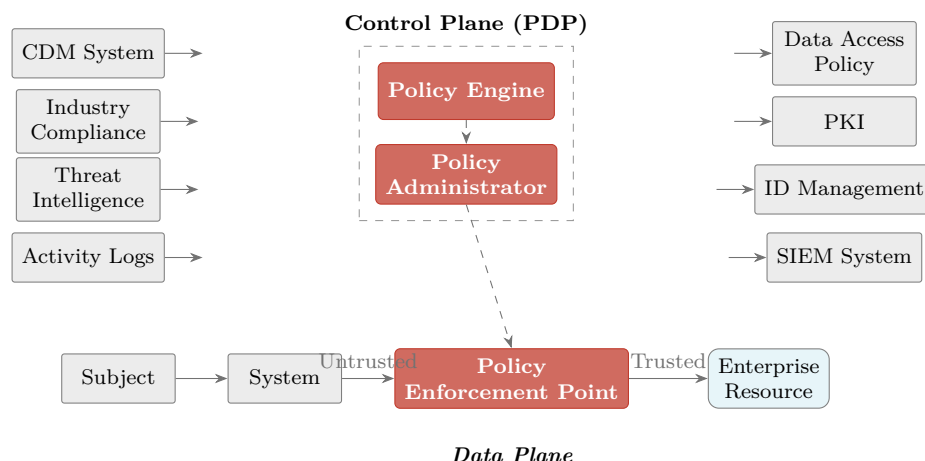
“te federo el login de una sola forma, por ejemplo a través de Active Directory, lo hacés una vez; después cada aplicación tiene los logins federados, no te pide un login nuevo, pero los

controles de seguridad se aplican porque se heredan los permisos y roles de esa persona en cada pieza.”

— dicho en clase

5.2.1 Componentes de ZTA (modelo NIST 800-207)

El núcleo separa un **Control Plane** (donde vive el **PDP** = Policy Decision Point, formado por el *Policy Engine* y el *Policy Administrator*) de un **Data Plane** (donde está el **PEP** = Policy Enforcement Point). El sujeto accede a través de un sistema considerado *untrusted*; sólo tras pasar el PEP el acceso al recurso es *trusted*.



Componentes que alimentan al motor de políticas

CDM — Continuous Diagnostics and Mitigation

Sistema de *diagnóstico y mitigación continuos*. Provee al motor de políticas información sobre el activo que solicita acceso: ¿tiene el antivirus actualizado? ¿los últimos updates de seguridad? Si una regla no se cumple, puede **desactivar el recurso** (ej.: software en la PC del empleado que no se puede desinstalar y, ante una anomalía, deshabilita la máquina).

Industry Compliance

Garantiza que la empresa cumpla los regímenes normativos aplicables: **HIPAA** (datos médicos), **Banco Central**, **SOX** (cotizar en bolsa, transparencia), **PCI** (datos de tarjeta de crédito).

Threat Intelligence

Fuentes de inteligencia sobre amenazas (internas/externas) que informan sobre nuevos ataques y vulnerabilidades (CVEs, hashes de malware, IPs comprometidas, botnets). Ej. de la slide: *VirusTotal*, feeds de IoC.

Activity Logs

Registros de actividad de la red y el sistema: tráfico, accesos a recursos y eventos que dan una visión (casi) en tiempo real de la postura de seguridad.

SIEM — Security Information and Event Management

Recolecta eventos y los **correlaciona** para su posterior análisis, detectar anomalías y generar alertas.

ID Management

Crea, almacena y gestiona las cuentas de usuario y los registros de identidad (nombre, email, certificados, función, privilegios y derechos).

“Yo soy Nando Suárez, pero en MELI soy lee.suarez: una persona puede tener varios usuarios.”
— dicho en clase

PKI — Public Key Infrastructure

Genera y registra los certificados emitidos por la empresa a recursos, sujetos, servicios y aplicaciones.

Data Access Policy

Características, reglas y políticas sobre el acceso a los recursos. Pueden estar codificadas (vía interfaz de gestión) o generarse dinámicamente por el PE, y son el punto de partida para autorizar el acceso a un recurso.

ZTA en el mundo real: todos los grandes proveedores publican guías de implementación — Azure, AWS, RedHat (Linux), Windows Server. La adopción de ZTA está en plena “oleada”.

Cuidado: ZTA mal implementado

ZTA parece la panacea, pero

“es muy fácil que lo implementemos mal.”
— dicho en clase

Casos típicos: implementarlo *de forma deficiente* (mal informados), o implementarlo y **no controlar nada** —“tenemos ZTA” pero no se monitorea ni se hace seguimiento de cómo evoluciona. Por eso son importantes las guías/framework y las **auditorías**.

“Si arrancás de cero, casi seguro conviene ir por ZTA, porque es arrancar sin ningún sesgo ni baggage.”
— dicho en clase

Migrar un Defense-in-depth existente a ZTA es muy difícil.

6 Privileged Access Management (PAM)

Privileged Access Management (PAM)

Se utiliza para **proteger y gestionar el uso de cuentas especiales** de una organización que tienen **acceso privilegiado** a sistemas críticos. Estos accesos son especialmente sensibles porque pueden: conceder a los usuarios capacidades amplias, modificar configuraciones de seguridad, acceder a datos confidenciales y supervisar otras cuentas de usuario.

Por qué los atacantes buscan estas cuentas

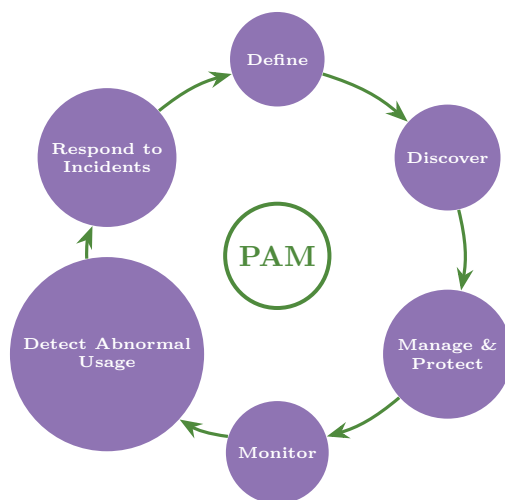
“Estos usuarios son lo que más buscan los atacantes, porque una vez que te haces con uno de ellos ya tenés allanado el camino para hacer daño, robar cosas, y tenés menos trabas para ejecutar tus acciones.”
— dicho en clase

6.1 Prácticas y herramientas de PAM

1. **Inventario de cuentas privilegiadas.** Inventariar todas las cuentas con acceso privilegiado para asegurar *visibilidad y control*; documentarlas y registrarlas.

2. **Principio de privilegio mínimo.** Los usuarios deben tener el mínimo acceso necesario para su trabajo. La cuenta privilegiada **no es la cuenta de trabajo diario**: es una cuenta separada, que puede transferirse cuando la persona se va o cambia de rol (de ahí que se reutilicen, lo cual tiene un lado malo).
3. **Autenticación multifactor (MFA).** 2, 3 o más factores para garantizar que sólo usuarios autorizados accedan, y **el no repudio** (nadie puede negar haber actuado: “vos tenías el TOTP, sólo vos podías avanzar sobre esos factores”).
4. **Registro y monitoreo.** Se registran y monitorean todas las actividades —en un servidor externo, guardando logs de auditoría— para detectar actividad sospechosa y poder *reconstruir lo que pasó*.
5. **Rotación de contraseñas.** Las contraseñas se cambian regular y automáticamente para reducir el riesgo de *compromisos prolongados*. También se hacen **reválidas**: cuando una cuenta se transfiere se cambia la contraseña, y cada tanto se revalida que la persona deba seguir teniéndola.

6.2 Ciclo de vida de PAM (PAM Life cycle)



El ciclo (según la slide): **Define** → **Discover** → **Manage & Protect** → **Monitor** → **Detect Abnormal Usage** → **Respond to Incidents** → **Review & Audit** (y vuelve a empezar). Resumido por el profe como: *descubrir, manejar y proteger, monitorear, detectar usos anormales, respuesta a incidentes, revisar y auditar*.

7 Prevención de vulnerabilidades: DevSecOps

Muchas organizaciones **construyen su propio software** y operan a través de él. A diferencia de los productos que se compran (donde el proveedor vela por las vulnerabilidades), aquí *nosotros* debemos gestionar y prevenir las vulnerabilidades de nuestro código.

De DevOps a DevSecOps (la historia que contó el profe)

Antes (80s/90s/2000s): el equipo de *desarrollo* programaba y un equipo de *operaciones* (*Ops*) desplegaba y monitoreaba —aislados y con **objetivos opuestos**: desarrollo quería sacar features rápido, operaciones quería no tocar nada para no romper la estabilidad.

“Se bloqueaba todo, había peleas, enfrentamientos.”

— dicho en clase

DevOps: alguien dice “estos equipos ahora son uno solo”, con el mismo objetivo: ser responsables de todo lo que se despliega y de que funcione. De ahí surgen CI/CD, infraestructura como código y las *métricas* (New Relic, Datadog, Grafana). Lema:

“you build it, you run it”
— dicho en clase

DevSecOps: se aplica el mismo principio al *equipo de seguridad*, que también estaba aislado y quería “frenar todo”. La idea: meter la seguridad en **cada paso** del proceso.

DevSecOps = Development + Security + Operations

Metodología que integra la seguridad en *todos* los procesos de desarrollo de software y operaciones de TI. La idea central es el **Shift Left**: meter la seguridad cada vez más “a la izquierda” del proceso (desde la planificación), porque **cuanto más temprano se detecta un problema, menos costoso es resolverlo**.



Security Champions: cómo escalar seguridad en cada equipo

El equipo de seguridad puede ser “10 personas vs. 1500 desarrolladores”. ¿Cómo estar en cada equipo? Con el programa **Security Champions**: se delega en una persona de cada equipo el rol de “campeón de seguridad”. No es trivial (hay que capacitar, y el expertise no es el de un especialista).

“DevSecOps implica que haya gente de seguridad en cada uno de los equipos desde el primer momento en que se toma un desarrollo y se planifica.”
— dicho en clase

Muchas empresas dicen tener DevSecOps cuando en realidad sólo hacen herramientas que le piden usar a los desarrolladores.

7.1 Las 5 fases de DevSecOps

1. Plan & Develop (Planeamiento y Desarrollo)

Aplicar todos los principios de seguridad vistos en la materia.

- **Threat modeling:** evaluar las amenazas/riesgos de una solución o feature nuevo *antes* de desarrollarlo (nueva superficie de ataque).
- **IDE security plugins** y uso de asistentes de IA para código seguro.
- **Pre-commit hooks:** forzar controles *antes* del commit, corriendo localmente (que no se expongan secretos ni vulnerabilidades).
- **Secure coding standards**, capacitación, **revisión de PRs** (peer review) y revisión de vulnerabilidades/correcciones.

2. Commit the code (On Commit/PR)

Controles **automatizados** antes de que el código siga el deployment.

- **SAST** (Static Application Security Testing): análisis estático del código (ej. SonarQube), que marca *code smells*, malas prácticas y problemas de seguridad.

- **Security unit & functional tests:** unit tests específicos de seguridad, con foco en los **CWE**; probar inyecciones y vulnerabilidades vistas en clase.
 - **Dependency management:** control de vulnerabilidades en las dependencias.
 - **Secure pipelines:** proteger el acceso a recursos confidenciales y claves de seguridad. Los pipelines definen, como antes la *cobertura de testing*, ahora un *nivel de seguridad* del feature.
- 3. Build & Test (On Build/Test, ambientes bajos / sandbox)** ▪ **DAST** (Dynamic Application Security Testing): controles dinámicos sobre la app *ejecutándose*, vía sus interfaces (APIs, URL endpoints).
- **Cloud configuration validation e Infrastructure scanning:** validar el código que crea recursos en la nube (IaC) y controlar la configuración tras correr el IaC.
 - **Security acceptance testing:** evaluación de seguridad de la aplicación como un todo; ejecución de ataques (mini pen-tests automatizados).
- 4. Go to Production (On Deployment)** ▪ **Security smoke tests y configuration checks** sobre los artefactos consumidos en el despliegue (configuraciones, secretos). Las configuraciones de producción difieren de las de ambientes bajos.
- Evaluación de la app *incluyendo el ambiente de despliegue* (Firewalls, WAF, IPS).
 - **Live site penetration testing** (pen-tests externos) y aparición de los **Red / Blue Team**.
- 5. Operate (On Operation)** ▪ **Continuous monitoring** de métricas y logs de seguridad.
- Integración con **Threat Intelligence**.
 - **Blind Penetration Test**, ejercicios de simulación de incidentes y **respuesta ante incidentes**.
 - **Blameless postmortems**.

El estado real de la industria

“Hoy la gran mayoría de las empresas tiene la seguridad atacando esta última etapa (operación): “ya se desarrolló, ya se testeó, ya está, ahora quiero ver la seguridad”. La idea es que la seguridad surja desde el principio. Cuanto más al principio, mejor.”

— dicho en clase

7.2 Testeos de seguridad en el código

DAST / SAST / SCA (la práctica los agrupa explícitamente)

DAST — Dynamic Application Security Testing

Evalúa la seguridad de la app **mientras se ejecuta**, interactuando como un usuario externo. Identifica vulnerabilidades explotables en la red (validación de entrada, autenticación). Usa las interfaces estándares de la app; se corre en ambientes bajos y **automatiza pruebas repetibles** (escanear puertos, inputs inválidos, requests que generan errores, clásico *Stack trace* expuesto).

SAST — Static Application Security Testing

Evalúa la app **analizando el código** (estático), sin ejecutarla.

SCA — Software Composition Analysis

Analiza las **dependencias y bibliotecas** usadas para identificar vulnerabilidades de seguridad, problemas de *licencia* y otros riesgos (incluidas dependencias *transitivas*). Debe extenderse a los servicios SaaS consumidos (bases de datos online, versiones de APIs, ambientes cloud).

Log4Shell: por qué importa el SCA y el SBOM

“A mí me tocó vivir la de Log4Shell.”

— dicho en clase

Vulnerabilidad crítica (2021) que permitía **ejecución remota de código** a través de *Log4j*, una librería de logging usada por casi cualquier aplicación Java del planeta.

“Salían todas las PoC [...] cosas impresionantes, y todos salieron a correr a ver qué tenía su aplicación, porque nadie sabía si realmente estaba usando Log4j: era una dependencia tan estándar que uno se olvidaba que la tenía.”

— dicho en clase

El parche salió rápido, pero *todas* las apps del mundo tuvieron que actualizar y desplegar. Moraleja: sin un inventario (SBOM) no sabés si estás afectado.

7.3 Inventario de software: SBOM

SBOM — Software Bill of Materials

Inventario **formal y legible por máquina** de los componentes de software y dependencias, con información sobre esos componentes y sus relaciones.

- Debe ser lo **más completo posible** e indicar explícitamente dónde no se pueden articular las relaciones.
- Incluye software de código abierto y con licencia comercial; puede estar ampliamente disponible o tener acceso restringido para proteger info confidencial.
- Componentes típicos (slide): Libraries, Software Components Name, Author Name, Supplier Name, Licences, Version String, Open Source Components, Dependencies, Compliance Requirements.

La historia del SBOM

Es una idea *vieja* (un inventario formal de componentes). Se dejó de mantener en cierto momento, y muchas empresas terminaron **sin saber qué componentes usaban** (de ahí el desastre Log4Shell). Hoy **resurge** por los incidentes recientes; hay plugins en los IDEs que detectan qué usás, si quedó obsoleto o tiene vulnerabilidades.

8 Protección de la nube

Ya se vio el **modelo de seguridad compartida** de la nube: la **configuración es responsabilidad del cliente**. El uso de **infraestructura como código (IaC)** permite definir la configuración de la nube de forma **reproducible**.

```
resource "aws_instance" "example_server" {
  ami = "ami-04e914639d0cca79a"
  instance_type = "t2.micro"

  tags = {
    Name = "JacksBlogExample"
  }
}
```

Listing 1: Ejemplo de IaC con Terraform (de la slide)

Configuraciones que quedan expuestas

“Hay redes que quedaron expuestas a la nada, a Internet [...] estás exponiendo puertos que no se están usando o demasiados puertos.”

— dicho en clase

Como IaC ya tiene varios años y está madurando, aparecen herramientas que detectan **patrones inseguros** y los marcan. Estas herramientas están pensadas para empresas con *miles y miles de servidores*; no las usa una Pyme.

Familias de herramientas de protección cloud

CSPM — Cloud Security Posture Management

Busca **problemas de configuración y permisos**. Analiza IaC *antes* de que sea desplegado y revisa la configuración actual para ver que no haya habido cambios. Enfoque más **estático**. (Analiza Infrastructure-as-Code y la valida.)

CWPP — Cloud Workflow/Workload Protection Platform

(en las slides aparece como “CWSS”). Se focaliza en la **protección de las aplicaciones corriendo** en la nube: servidores virtuales, contenedores y *serverless* (ej. Lambdas). Analiza comportamiento, microsegmentación a nivel de contenedores, threat intelligence. **Complementa a CSPM** (que es más estático).

CIEM — Cloud Infrastructure Entitlement Management

Foco en los **permisos de los sujetos sobre los recursos** de la nube. Busca **permisos excesivos**: compara los registros de uso de los accesos contra los permisos otorgados para identificar y remover los no utilizados. Muestra consistencia entre cuentas cloud y permite una visualización efectiva de permisos (útil con múltiples cuentas).

CNAPP — Cloud Native Application Protection Platform

La “sombrija” que integra todo lo anterior. Vista simplificada (Gartner): combina *seguridad proactiva* (development artifact scanning) con *reactiva* (runtime visibility/protection), atravesada por DevSecOps y *cloud configuration and compliance*.

9 Evaluaciones de seguridad

¿Por qué evaluar la seguridad?

“Construimos y tratamos de hacer nuestro sistema seguro, tomamos todas las precauciones y las mejores prácticas... y en algún momento ese sistema va a fallar. No pregunten cuándo, no pregunten cómo, pero va a fallar, así hayamos tomado todos los recaudos. ¿Por qué? Porque la seguridad no está asegurada.”

— dicho en clase

Van surgiendo nuevos vectores de ataque que uno no contempla; por eso hay que estar **evaluando todo el tiempo**: es un proceso constante y sin pausa.

“Security doesn’t have to last forever; just longer than everything else that might notice it’s gone.”
(cita de la slide)

9.1 Tipos de evaluaciones de seguridad

Las 6 evaluaciones de seguridad

Evaluación de Vulnerabilidades

Uso de **herramientas automatizadas** para escanear sistemas y redes en busca de *vulnerabilidades conocidas*. Da un resumen de debilidades explotables.

Pruebas de Penetración (PenTest)

Más **exhaustivas**; realizadas por expertos que intentan *explotar vulnerabilidades de manera controlada*. Ayudan a entender cómo un atacante podría acceder.

Evaluación de Riesgos

Identificar, evaluar y priorizar riesgos para los activos de información. Visión integral (la seguridad como una amenaza más del negocio).

Auditoría de Seguridad

Revisión *sistemática* de políticas, procedimientos y controles. Interna o externa; busca asegurar el cumplimiento de normativas y estándares.

Evaluación de Cumplimiento

Verifica que la organización cumpla regulaciones y estándares aplicables: **GDPR, HIPAA, PCI-DSS**. Obligatoria para ciertas regulaciones.

Evaluación de Seguridad Física

A menudo se pasa por alto pero es crucial: control de acceso a instalaciones, cámaras, protección de archivos físicos y equipos.

Evaluación de Seguridad de Aplicaciones

Vulnerabilidades en software y apps web: revisiones de código, pruebas de seguridad y análisis de arquitectura.

9.2 Penetration Test (PenTest)

Objetivo del PenTest

Agarrar el **sistema funcionando** y probarlo integralmente para **identificar vulnerabilidades** que permitan que el sistema pase de un **estado seguro a uno inseguro**.

“Muchas veces el diseño es bueno y seguro, pero si le pasás ciertos inputs o ante cierto comportamiento, deja de ser seguro.”

— dicho en clase

- Se realizan con un **alcance definido**, en **tiempo y sistemas evaluados** (podrían extenderse indefinidamente).
- Pueden hacerse **metódicamente**, como control recurrente para verificar que los controles definidos siguen en su lugar.
- Pueden ejecutarlos un **grupo de expertos** (internos o un proveedor contratado), para identificar problemas no detectados antes.

9.2.1 Tipos de PenTest según visibilidad del evaluador

Tipo	Descripción
Black Box	El evaluador <i>no tiene conocimiento previo</i> del sistema (a lo sumo el dominio o IP). Simula un atacante externo sin info interna. También llamado Blind Test .
White Box	El evaluador tiene <i>acceso completo</i> a la info interna: código fuente, diagramas de red, credenciales. Evaluación exhaustiva de vulnerabilidades internas.
Gray Box	Enfoque <i>intermedio</i> : algún conocimiento, sin acceso completo. Simula un usuario con acceso limitado (empleado con privilegios restringidos).
Double Blind	Es un ataque Black Box donde <i>el atacado tampoco está al tanto</i> . Evalúa adicionalmente las capacidades de monitoreo y respuesta . Es lo más parecido a la realidad.

¿Por qué Black Box si White Box parece más eficiente?

Pregunta de un alumno. Respuesta del profe:

“El Black Box te da descubrir vectores que vos, estando sesgado con toda la información, no verías. Estar afuera con la presión de “tengo que lograr entrar” te da un montón de creatividad; con toda la info te metés en la piel del desarrollador y te terminás encerrando en esa caja sin poder salir.”

— dicho en clase

Además, la razón más clara para Black Box es que **no querés compartir** info sensible (red, credenciales, código fuente) al proveedor, muchas veces por motivos *legales*.

“Generalmente se suelen hacer de los dos tipos.”

— dicho en clase

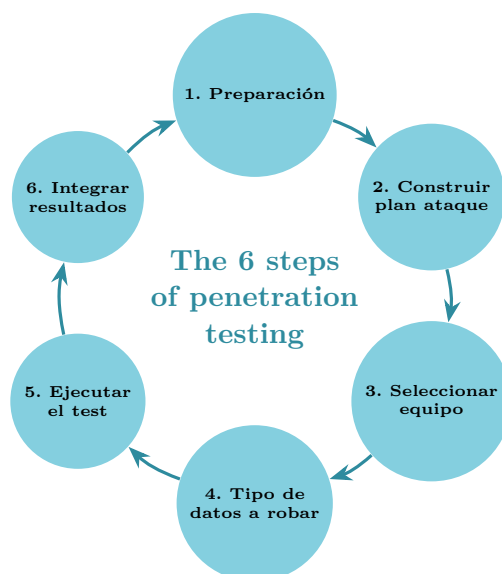
Double Blind y Chaos Monkey

El *Double Blind* se parece a la metodología **Chaos Monkey**: ir rompiendo a propósito aplicaciones dentro de la infraestructura (interrupciones de servicio) *sin avisar* y ver cómo reacciona el equipo. Si está bien focalizado y controlado, genera aprendizaje sin bajar la operación.

9.2.2 Tipos de PenTest según el sistema evaluado

- **Test de Red Externa:** sistemas y servicios expuestos a Internet (atacantes externos).
- **Test de Red Interna:** red interna; simula un ataque desde dentro (empleado descontento o malware en una workstation).
- **Test de redes inalámbricas:** NFC, Bluetooth, Wireless, entre otras.
- **Test de Aplicaciones Web:** inyecciones SQL, XSS, etc.
- **Test de Ingeniería Social:** susceptibilidad de los empleados a ser engañados (phishing, smishing, campañas con supuestos regalos).

9.2.3 Etapas de un PenTest (6 pasos)



1. **Preparación.** Identificar el objetivo a evaluar, el alcance y **acordar el nivel de riesgo** que la organización desea correr (ej.: un ataque de fuerza bruta podría dejar cuentas bloqueadas y afectar sistemas). Entender legalidades y propósito.
2. **Armado del plan de ataque.** Definir herramientas a utilizar y la ubicación lógica y física desde donde se lanzará.
3. **Identificar el equipo de trabajo.** Expertos en distintas tecnologías y generalistas con conocimiento amplio.
4. **Definir el objetivo a alcanzar.** Estos ataques son muy enfocados; muchas veces *no aportan* info sobre la efectividad del sistema como un todo si no logran el objetivo.

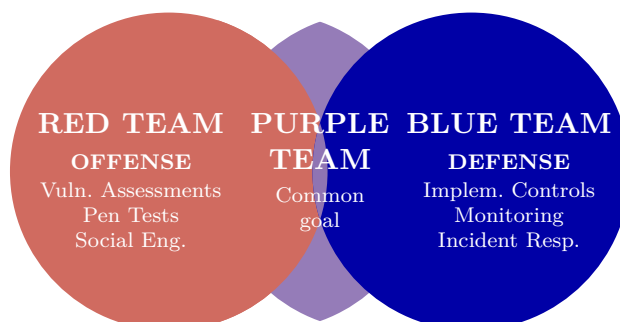
“Es mejor no tener éxito, porque eso implica que no hay que hacer cambios.”

— dicho en clase

5. **Ejecutar el ataque.** Bajo el tiempo y alcance definidos. Termina al lograr el objetivo o al vencerse el tiempo.
6. **Integrar los hallazgos** y reiniciar el ciclo, *pivoteando* y escalando con lo encontrado. Ejemplo del profe: (1) ingeniería social → (2) ataque de phishing → (3) con las credenciales se accede al repositorio de archivos y se extraen documentos → con esos documentos se prepara el siguiente ataque.

9.3 Red / Blue / Purple Team

Son divisiones dentro del equipo de seguridad de la organización.



Red / Blue / Purple Team

Red Team (ofensivo)

Piensa y actúa como un **atacante malintencionado**, usando técnicas de hacking para comprometer sistemas, redes y aplicaciones *de forma ética y controlada*. Ejecuta pen-tests, ingeniería social y assessments de vulnerabilidades, para probar la efectividad de las medidas y mejorar la detección y respuesta.

Blue Team (defensivo)

Implementa y mantiene medidas de seguridad (firewalls, IDS, gestión de parches y configuraciones), **monitorea** en busca de actividad sospechosa y **responde** a intentos de intrusión. Objetivo: proteger los activos y mitigar riesgos de ciberataques.

Purple Team (colaborativo)

Enfoque que **combina** los esfuerzos del Red y el Blue Team para mejorar la postura de seguridad. En lugar de trabajar aislados, fomenta la comunicación y colaboración entre ofensiva y defensiva.

“Muchas veces surge de la pelea entre ambos teams.”

— dicho en clase

9.4 Breach and Attack Simulation (BAS)

Breach and Attack Simulation (BAS)

Categoría de herramientas y técnicas para **evaluar la efectividad de las defensas** de una organización contra posibles ataques. Simulan las **tácticas, técnicas y procedimientos (TTP)** que un atacante real podría emplear.

- Objetivo: identificar vulnerabilidades y brechas *antes* de que las exploten atacantes reales, y medir la **capacidad de la organización para identificar y reaccionar** ante ataques.
- Se evalúan las *mismas herramientas* que protegen la organización y los sistemas de monitoreo y alertas; identifican cambios de configuración o procesos que dejen de actuar como se espera.
- Se ejecutan de **forma recurrente**, probando en distintos puntos la capacidad de detección, mitigación y respuesta.

“War games”: baratos pero valiosos

El profe lo asocia a los *juegos de guerra*: se simula un escenario desfavorable puntual, p. ej.

“¿qué pasa si un atacante obtiene acceso a una ruta/credencial de admin del cloud? Tiene la ruta, puede borrar infraestructura, crear nueva, hacer lo que quiera.”

— dicho en clase

Se analiza qué tácticas y técnicas usar para defenderse y qué equipos se involucran.

“Son baratos entre comillas: solamente juntar a la gente involucrada y proponer el escenario.”

— dicho en clase

A veces se prueba incluso en un ambiente controlado.

Resumen visual: el panorama de la clase



Lectura recomendada

NIST SP 800-207 — Zero Trust Architecture

Capítulos 2 y 3

<https://csrc.nist.gov/pubs/sp/800/207/final>

Nota de alcance. El profe aclaró que *esta clase no está en Bishop*: el libro tiene material relacionado, pero esta clase está enfocada en el panorama profesional contemporáneo y se apoya en el estándar **NIST 800-207**. El detalle del *Penetration Testing* (metodología paso a paso) corresponde además a la clase aparte de **Pentesting (Clase 10b)**.